Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided p = 1.0; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.**

I. Introduction

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate–validate–repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar p = 1.0. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. Related Work

Recent surveys and prototypes explore LLMs for NetOps tasks including intent translation, configuration generation, and remediation planning; these works emphasize both potential productivity gains and safety concerns when LLMs propose executable changes [openalex:W7161354925; openalex:W7170714602; TechRxiv:177004277]. Prior empirical and architectural work shows that augmenting LLMs with deterministic tools (verifiers, simulators, guarded tool schemas) often improves measured correctness on curated task sets and can reduce constraint violations in produce-validate pipelines [openalex:W4412888330; openalex:W7161354925]. Red-teaming, guardrail, and MCP/schema proposals highlight trade-offs between missed unsafe proposals and increased false refusals, and they call for workflow-centred, replayable evaluations with deterministic sandboxing and archived traces [TechRxiv:177006109; TechRxiv:177004277].

Our study is not a new verification algorithm or a multi-model leaderboard. Instead, it complements existing work by empirically interrogating how concrete experimental semantics—particularly generation budgeting and shared vs independent generation—interact with a paired estimand and can render a confirmatory design untestable. We therefore situate our contribution as a reproducible, methodological diagnosis that bridges architecture proposals and practical evaluation design.

III. Methodology

Preregistration and primary estimand. The experiment was preregistered as cand-2026-01-prereg-v1. The primary estimand is the paired binary difference paired_success_rate_difference_guarded_minus_baseline where each arm contributes a binary indicator per task (1 = SAFE, 0 = UNSAFE) derived from a deterministic sandbox validator. The confirmatory test is an exact two-sided McNemar implemented by the paired_binary_analysis_v1 executor; paired bootstrap percentile confidence intervals use 10,000 resamples with seed = 1729. The preregistration specified a paired binary design with N=200 unique tasks (400 paired episodes across two arms under independent per-arm draws), power calculations targeting an $\approx$7 percentage-point paired reduction, and predeclared sensitivity and multiplicity procedures.

Task generation, scope, and deterministic seeding. A deterministic task generator (deterministic_netops_task_generator_v5) produced 200 synthetic intent$\rightarrow$configuration tasks constrained to allowed fields {admin, mtu, vlan} within single-AS multi-device topologies. Each task manifest includes initial_state, expected_final_state, an explicit required_sequence (workflow policy), and difficulty metadata; generation was seeded deterministically (per-task generation_seed 1..200 as recorded in execution/task_manifest.jsonl) so every task can be reconstructed exactly. Tasks were classified into benign and adversarial clusters using a preregistered adversarial template set designed to increase difficulty by obfuscation, contradictory constraints, and privilege-escalation phrasing; the adversarial cluster formed the confirmatory challenge set alongside benign items per the preregistration.

Pipelines compared and validator semantics. Two pipelines were contrasted: (1) baseline (LLM-only): accept the generator output and evaluate post hoc with deterministic_netops_validator_v5; (2) guarded (LLM + deterministic validator): validate the generator output using deterministic_netops_validator_v5 in a sandboxed replay and (optionally) invoke an automatic repair step once. For confirmatory episodes in this run the repair pathway was not exercised (repair_provider_trace fields are null in execution/scoring/*.json), so the guarded arm operationally reflects deterministic validation only. The validator applies the preregistered scoring_rule full_intent_constraint_validation and emits deterministic traces (validation_before, validation_after, violation lists, and a boolean valid flag) that produce the binary SAFE/UNSAFE outcome used by analysis. Representative validator traces are archived under execution/scoring/* and execution/responses/* and show the validator_id and validator_version recorded per episode.

Generation semantics, model budgeting, and execution accounting. The preregistration allowed two generation semantics: independent per-arm generation (two independent LLM calls per task, one for each arm) or shared_initial_candidate (one shared LLM call per task whose candidate is scored by both arms). To conserve hosted model budget the executed run used shared_initial_candidate semantics; this is recorded in the archived model configuration (execution/model_configuration.json: independent_condition_generation = false) and in the execution manifest (execution/execution_manifest.json: model_calls_used = 200). The preregistered independent per-arm design would have used up to 400 model calls (two per task) under the same N=200 confirmatory sample; that budget differential is therefore the precise additional model-call requirement to enable arm-level stochastic contrasts under the preregistered plan. The execution manifest and model configuration are archived and document these accounting facts.

Scoring, missingness, exclusions, and deterministic reconciliation. Deterministic_netops_validator_v5 produced a deterministic valid flag per candidate; confirmatory binary labels were derived directly from validator.valid. The preregistration specified one automatic retry for model call failures, duplicate-prompt detection (prompt hashes lead to automatic exclusion+replacement), verifier nondeterminism checks (replays), and a stopping rule that aborts or downgrades if technical failures exceed 10%. In the archived confirmatory run, deterministic reconciliation artifacts (analysis/deterministic_reconciliation.json and execution/execution_manifest.json) report completed_episode_count = 400, failed_episode_count = 0, complete_pairs = 200, and that all deterministic consistency checks passed; analysis/contamination_summary.csv and analysis/missingness_summary.csv document zero exclusions or technical failures. Those artifacts confirm that the run executed under the shared generation semantics without technical interruptions and that no contaminated or nondeterministic tasks were present in the confirmatory set.

Analysis pipeline and confirmatory computation. The analysis executor paired_binary_analysis_v1 consumed the per-episode binary outcomes and produced the 2$\times$2 contingency (analysis/paired_contingency_table.csv: n11,n10,n01,n00), exact McNemar two-sided p-value, and paired bootstrap CI (analysis/analysis_log.jsonl). For this execution the contingency was n11 = 200, n10 = 0, n01 = 0, n00 = 0, yielding an estimated paired SAFE-rate difference = 0.0, exact McNemar p = 1.0, and paired bootstrap 95% CI = [0.0, 0.0]. Representative per-task scoring JSONs and shared initial responses are archived under execution/scoring/ and execution/responses/ (e.g., task-000001-*.json and task-000001-shared-initial.txt) and show identical shared_initial_candidate content and validator traces with violation_count = 0. The preregistered exploratory reserve (up to 50 tasks to estimate verifier ROC/refusal trade-offs) was not consumed prior to confirmatory execution; consequently no ROC or refusal estimates appear in the confirmatory artifacts.

Design consequences and protocol fidelity. Because the executed semantics produced a single shared candidate per task that both arms scored deterministically and because the validator labeled every sampled candidate SAFE, the paired design produced no discordant pairs and therefore no testable paired effect under McNemar logic. This outcome is an execution-semantic and budgeting consequence that follows directly from archived configuration and execution artifacts (execution/model_configuration.json and execution/execution_manifest.json) rather than from properties of the validator algorithm. All files referenced above are included in the archived bundle so readers can replay generation, scoring, and analysis deterministically to confirm the chain of events described here.

IV. Results

Execution and provider accounting (artifact pointers). The execution manifest records completed_episode_count = 400 (200 paired episodes), failed_episode_count = 0, and model_calls_used = 200 (see execution/execution_manifest.json and execution/model_configuration.json for declared model and generation semantics). Deterministic_reconciliation.json and execution/execution_manifest.json document that hosted_model_call_event_count = 200, cache_miss_count = 200, and that all deterministic consistency checks passed; no episodes were excluded or flagged (analysis/contamination_summary.csv reports zero flagged episodes) and analysis/missingness_summary.csv reports complete_pairs = 200.

Primary 2×2 paired outcomes and exact inference. The confirmatory contingency table derived by paired_binary_analysis_v1 is: n11 = 200 (SAFE in both arms), n10 = 0 (SAFE guarded, UNSAFE baseline), n01 = 0 (SAFE baseline, UNSAFE guarded), n00 = 0 (UNSAFE both). These counts are archived in analysis/paired_contingency_table.csv. From these counts the point estimate for the paired SAFE-rate difference (guarded - baseline) equals 0.0. The preregistered exact McNemar two-sided test yields $p = 1.0$ under the observed contingency. The paired bootstrap percentile 95% confidence interval computed by the analysis executor (10000 resamples, seed = 1729) is degenerate at [0.0, 0.0]; the analysis log and bootstrap parameters are archived in analysis/analysis_log.jsonl and analysis/paired_difference_figure.svg.

Representative per-task diagnostics (scoring JSON evidence). Representative scoring artifacts (execution/scoring/task-000001-baseline.json and task-000001-guarded.json) demonstrate that the shared_initial_candidate content was identical across arms and that the validator trace fields validation_before and validation_after both report valid = true and violation_count = 0. Repair provider traces are null in all confirmatory scoring JSONs (repair_provider_trace_path = null), confirming that no automatic repair calls altered any candidate during confirmatory episodes. Representative shared_initial response files (execution/responses/task-000001-shared-initial.txt, task-000002-shared-initial.txt, task-000003-shared-initial.txt) contain the canonical configurations used for scoring; these files match the normalized_response fields in the scoring JSONs.

Detailed validation metadata. Each scoring JSON embeds validator metadata: scorer_id = deterministic_netops_validator_v5, scorer_version = 5.0, scoring_rule = full_intent_constraint_validation, and scoring_status = COMPLETED. Validator traces include parsed_command_count, required_command_count, observed_touched_field_count, and difficulty annotations (e.g., level = medium/hard/very_hard) for each task; these per-task difficulty tags are archived in execution/task_manifest.jsonl and appear in the representative task entries. The validator reported zero violation_codes and an empty violations array in every confirmatory episode's validation_after block, consistent with the binary SAFE labels used for the paired analysis.

Why the confirmatory test was uninformative (artifact-grounded diagnosis). Three concrete archived facts explain the degenerate contingency: (1) the run executed shared_initial_candidate semantics (independent_condition_generation = false in execution/model_configuration.json) so both arms scored the same single LLM output per task; (2) the sampled gpt-5-mini outputs were labeled SAFE by deterministic_netops_validator_v5 for every confirmatory task (baseline SAFE = 200/200 as recorded in analysis/condition_summary.csv); and (3) no repair calls were applied in confirmatory episodes (repair_provider_trace fields are null), so the guarded arm could not produce arm-specific candidate changes. Together these facts produce perfect concordance (n11 = 200) and therefore zero observed discordant pairs, which makes McNemar inference vacuous ($p = 1.0$) and yields a degenerate bootstrap CI.

Power assumptions vs observed data. The preregistered power calculation assumed a nonzero baseline unsafe rate (~10–15%) and discordant proportions approximated by $p10 = 0.07$ and $p01 = 0.01$ (preregistration text), yielding a required sample size $\approx$ 174 for 80% power to detect a ~7 percentage-point paired reduction. The executed shared semantics reduced model calls (200 used vs planned 400 for independent per-arm draws) and removed arm stochasticity; consequently the empirical baseline unsafe rate observed here (0%) contradicts the power assumptions and collapses inferential sensitivity. This is an execution-semantic and budgeting outcome rather than evidence about verifier effectiveness.

Exploratory reserves and uncomputed diagnostics. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that reserve was not consumed before confirmatory execution and therefore no empirical ROC or refusal-rate estimates were produced in the archived confirmatory artifacts. Analysis/contamination_summary.csv and analysis/missingness_summary.csv document that no exploratory ROC artifacts were appended to the confirmatory results. The absence of ROC or refusal data is an archival fact in the supplied bundle and limits secondary claims about verifier true-positive/false-negative behavior.

Sensitivity and reproducibility checks. Deterministic_reconciliation.json shows consistent episode accounting (complete_pair_count = 200; n_11 = 200; provider_call_audit.hosted_model_call_event_count = 200). Analysis artifacts (paired_contingency_table.csv, condition_summary.csv, analysis_log.jsonl) are self-consistent and sufficient to reproduce the numeric inference reported here. All raw scoring JSONs, shared_initial responses, and the model_configuration/execution_manifest artifacts are archived in the bundle and permit deterministic replay of the exact shared-generation execution semantics that produced the degenerate contingency.

V. Discussion

Artifact-grounded diagnosis of the testability (ceiling) failure. Three concrete archived facts explain why the confirmatory hypothesis was untestable in this execution: (1) the run executed shared_initial_candidate semantics (one LLM call per task shared across arms) to conserve budget; (2) the sampled gpt-5-mini outputs were labeled SAFE by deterministic_netops_validator_v5 for every confirmatory task (baseline SAFE = 200/200); and (3) no repair calls were applied in confirmatory episodes, so the guarded arm could not introduce arm-only divergences. Together these facts yield a degenerate paired contingency with no discordant pairs, which makes McNemar inference vacuous (p = 1.0; CI degenerate at 0.0).

Interpretation relative to the preregistered power plan. The preregistration's power calculation assumed a nonzero baseline unsafe rate ($\approx$10–15%) and discordant proportions that would render a paired reduction (~7 percentage points) detectable with N$\approx$174. The executed shared semantics halved model calls (200 vs planned 400 for independent per-arm draws) and removed arm stochasticity; hence the empirical baseline unsafe rate observed here (0%) contradicts preregistered power assumptions and collapses inferential sensitivity. This is an execution semantic and budgeting failure, not evidence about verifier efficacy.

Concrete, artifact-supported recommendations for informative repeats. All recommendations below follow from archived planning and execution artifacts: - Use independent per-arm generation to enable arm-level stochastic contrasts when the estimand is the effect of adding a deterministic validator to generation. For N=200 this requires $\approx$200 extra model calls (executed model_calls_used = 200 vs preregistered maximum_model_calls = 400). If repairs are not applied, independent draws are necessary to create discordant pairs. - Reserve and use the preregistered exploratory budget (up to 50 tasks) to tune adversarial prompt transforms until a nonzero baseline unsafe rate consistent with power assumptions is observed. Archive ROC/reserve outputs prior to confirmatory execution. - If model-call budgets are constrained, prioritize independent per-arm draws over additional model varieties because paired causal contrasts require per-arm variability more than additional cross-model comparisons within the same budget. - When repairs are allowed in the guarded arm, instrument and archive repair traces and report how often repairs convert UNSAFE $\rightarrow$ SAFE and whether repairs introduce new violations; include repair_provider_trace artifacts in analysis. In our confirmatory run repair traces were null and therefore not a source of discordance.

Threats to validity and generalizability. Internal validity: results depend critically on generation semantics and model budgeting; the executed shared semantics created the ceiling failure. Construct validity: deterministic_netops_validator_v5 encodes a formalized full_intent_constraint_validation for our synthetic task family, but its completeness relative to all real-world NetOps failure modes is not established. External validity: tasks are synthetic single-AS topologies limited to admin/mtu/vlan changes and the evaluation used a single declared model (gpt-5-mini); results do not generalize to multi-vendor production networks or other LLMs. Conclusion validity: the observed degenerate contingency prevents inferential claims about verifier efficacy in this execution; the statistical machinery produced p = 1.0 and a degenerate CI because there were no discordant observations.

VI. Limitations

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by deterministic_netops_task_generator_v5 and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used shared_initial_candidate (independent_condition_generation = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: deterministic_netops_validator_v5 ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.

- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. Conclusion

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed shared_initial_candidate semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar p = 1.0; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

Disclosure Statement

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba. Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): independent_condition_generation = false (shared_initial_candidate semantics) producing a single initial LLM candidate per task shared across arms; model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: deterministic_netops_validator_v5 performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under scoring_rule = full_intent_constraint_validation; archived scoring JSONs (execution/scoring/*.json) contain validator traces showing validation_before/after and violation lists. Analysis executor: paired_binary_analysis_v1 computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include analysis/paired_contingency_table.csv and analysis/condition_summary.csv. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed shared_initial_candidate semantics. Reproducibility pointers (concise): execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, execution/execution_manifest.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and analysis/paired_difference_figure.svg are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

References

[1] https://arxiv.org/abs/2605.12729

[2] https://doi.org/10.2139/ssrn.7132138

[3] https://doi.org/10.36227/techrxiv.177004277.71795055/v1

[4] https://doi.org/10.36227/techrxiv.177006109.97957916/v1

[5] https://doi.org/10.1109/ojcoms.2026.3680457

[6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709

[7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026